

RAPPORT DE PROJET

Animations Manim pour la topologie métrique

Une bibliothèque Python pour la visualisation des notions du cours de topologie de licence 3

Auteurs **Solveig GIRARDIN**
 Walid KHALFALLAH
 Anis LAPLAGNE
 Yuyang ZHOU

Formation LU3MA201 — Projet / travail d'étude et de recherche, Licence 3 Mathématiques
Organisme Sorbonne Université
Encadrement Frédéric Le Roux
Référence Sorbonne Université
Période Printemps 2025

Remerciements

Nous tenons à exprimer notre reconnaissance à l'équipe pédagogique du module **LU3MA201** pour l'encadrement attentif dont nous avons bénéficié tout au long de ce projet. Nous adressons en particulier nos remerciements à **M. Frédéric Le Roux**, qui a accepté de superviser ce travail et dont les conseils ont grandement contribué à préciser nos objectifs scientifiques, à consolider notre compréhension des notions topologiques abordées et à orienter nos choix de représentation visuelle. Notre gratitude va également à l'ensemble du département de mathématiques de Sorbonne Université, qui met à la disposition des étudiants un cadre propice à la conduite de projets tels que celui-ci.

Ce rapport n'aurait pu voir le jour sans les travaux préparatoires de **F. Le Roux** et **F. Klopp**, dont le *Mémo de topologie* (2021) constitue la référence principale sur laquelle s'appuie l'ensemble de nos scènes. Nous saluons enfin le travail de la communauté *Manim*, dont la documentation et les exemples publics ont rendu possible la réalisation technique de ce projet.

Résumé

Résumé

Le présent rapport rend compte d'un projet de visualisation mathématique mené dans le cadre du module **LU3MA201** de la licence de mathématiques de Sorbonne Université. Partant du constat que les concepts de la topologie métrique — connexité, compacité, complétude — mobilisent une intuition géométrique que le seul formalisme peine parfois à transmettre, nous avons entrepris de construire, à l'aide de la bibliothèque Python *Manim*, un ensemble d'animations pédagogiques fidèles au cours de topologie de licence 3. Le travail s'est articulé autour de deux axes complémentaires : d'une part, la conception d'une *bibliothèque interne* factorisant les primitives graphiques et les conventions chromatiques communes à l'ensemble des scènes ; d'autre part, la réalisation de cinq scènes autonomes couvrant la connexité, la compacité de Borel–Lebesgue et le théorème de Baire. Le rapport expose la démarche méthodologique adoptée, détaille l'architecture logicielle du dépôt, analyse en profondeur une scène représentative et discute des choix de modélisation, tant sur le plan mathématique que sur le plan graphique.

Mots-clés : *Manim*, topologie métrique, connexité, compacité, complétude, théorème de Baire, animation mathématique, visualisation, Python, pédagogie des mathématiques.

Abstract

This report documents a mathematical visualisation project carried out as part of the **LU3MA201** module in the Mathematics Bachelor's programme at Sorbonne Université. Starting from the observation that the concepts of metric topology — connectedness, compactness, completeness — rely on a geometric intuition that formal language alone sometimes fails to convey, we have designed, using the Python library *Manim*, a coherent set of pedagogical animations faithful to the third-year topology course. The work is organised along two complementary axes : on the one hand, the design of an *internal library* that factorises the graphical primitives and chromatic conventions shared by all scenes ; on the other hand, the production of five self-contained scenes covering connectedness, Borel–Lebesgue compactness and Baire's theorem. The report sets out the methodology adopted, details the software architecture of the repository, offers an in-depth analysis of a representative scene and discusses the modelling choices, both mathematical and graphical.

Keywords : *Manim*, metric topology, connectedness, compactness, completeness, Baire's theorem, mathematical animation, visualisation, Python, mathematics education.

Table des matières

Remerciements	2
Résumé	3
1 Introduction	8
1.1 Contexte et motivation	8
1.2 Cadre du projet	8
1.3 Périmètre effectif du travail	9
1.4 Plan du rapport	10
2 Architecture du projet	11
2.1 Principe directeur : séparer ce qui est réutilisable de ce qui est singulier . .	11
2.2 Lecture architecturale en couches	11
2.3 Couche d’environnement : définir sans ambiguïté le cadre d’exécution . .	12
2.3.1 Le fichier <code>pyproject.toml</code>	13
2.3.2 Le <code>Makefile</code> : une interface unifiée de commandes	13
2.3.3 Le répertoire <code>tests/</code>	15
2.3.4 Les répertoires annexes	15
2.4 Couche de mutualisation : aperçu du répertoire <code>src/</code>	15
2.5 Couche de scénarisation : aperçu du répertoire <code>scenes/</code>	16
2.6 Cycle de vie d’une scène : de l’idée mathématique au fichier vidéo	16
2.7 Synthèse cartographique des principaux fichiers	17
2.8 Appréciation critique de l’état actuel	19
3 Anatomie d’une scène <i>Manim</i>	20
3.1 <i>Manim</i> comme langage descriptif de scènes	20
3.2 Un exemple fondateur : visualisation d’un ensemble ouvert	21

3.3	Lecture analytique de l'animation	23
3.4	Ce que l'exemple nous a appris	24
4	Bibliothèque interne	25
4.1	Configuration globale : le module <code>src/config.py</code>	25
4.2	Palette chromatique sémantique : <code>src/utils/colors.py</code>	25
4.3	Utilitaires de mise en page : <code>src/utils/layout.py</code>	27
4.4	Labels $\text{\LaTeX}$ standardisés : <code>src/utils/tex_labels.py</code>	28
4.5	Objets mathématiques visualisables : <code>src/objects/</code>	28
4.6	Schémas d'animation réutilisables : <code>src/animations/</code>	29
4.7	Remarque méthodologique	30
5	Les scènes produites	31
5.1	Vue d'ensemble	31
5.2	La scène <code>ConnexeVsArcs</code> : opposer deux intuitions	32
5.3	La scène <code>InvarianceTopologique</code>	33
5.4	La scène <code>ContreExempleSin1x</code>	34
5.5	La scène <code>BorelLebesgue</code>	34
5.6	La scène <code>Baire</code>	34
5.7	Cohérence pédagogique de l'ensemble	35
6	GitHub et Git	36
6.1	Rôle de Git et de GitHub dans un projet collaboratif	36
6.2	Le cycle élémentaire d'utilisation	37
6.3	Commandes fondamentales : une brève anthologie	37
6.3.1	Récupération ou initialisation du dépôt	37
6.3.2	Inspection de l'état local	38
6.3.3	Enregistrement d'une modification	38
6.3.4	Publication distante : <code>git push</code>	38
6.3.5	Mise à jour locale : <code>git pull</code>	39
6.3.6	Branches de travail	39

6.4	Application au dépôt topo-manim	39
6.5	La gestion de versions comme discipline académique	40
7	Étude de cas	41
7.1	Un cas d'étude exemplaire	41
7.2	Rappels mathématiques	41
7.3	Structure logicielle et imports	42
7.4	Analyse détaillée des sections	43
7.4.1	Section 1 : le titre	43
7.4.2	Section 2 : la compacité séquentielle	43
7.4.3	Section 3 : le recouvrement	44
7.4.4	Section 4 : le segment compact	44
7.4.5	Section 5 : l'exemple non compact	44
7.4.6	Section 6 : le lemme de Lebesgue	45
7.5	L'inscription du fichier dans l'architecture du dépôt	45
7.6	Logique pédagogique : une montée en abstraction ordonnée	46
7.7	Pourquoi ce fichier est un bon exemple	46
8	Conclusion et perspectives	48
8.1	Bilan du travail accompli	48
8.2	Difficultés rencontrées et enseignements	49
8.3	Perspectives d'extension	49
8.4	Mot de la fin	50
	Références bibliographiques	51

Introduction

1.1 Contexte et motivation

La topologie métrique occupe, dans la formation mathématique de licence, une place singulière : elle est à la fois le langage dans lequel se formulent les notions fondamentales d'analyse — limite, continuité, convergence — et la première véritable confrontation de l'étudiant à une pensée abstraite des objets géométriques. Les ensembles ouverts, les recouvrements, les suites convergentes ou les chemins continus sont, par nature, des objets géométriques ; pourtant, leur définition formelle peut paraître austère, voire contre-intuitive, à qui n'a pas encore construit l'imaginaire visuel correspondant. C'est précisément au franchissement de ce seuil intuitif que l'animation mathématique peut contribuer de manière décisive.

La bibliothèque *Manim* — acronyme de *Mathematical Animation Engine* — a été initialement développée par Grant Sanderson, créateur de la chaîne de vulgarisation mathématique [3Blue1Brown](#), afin de produire les animations qui ont rendu cette chaîne célèbre pour la clarté de ses exposés [2]. Maintenue et enrichie depuis par une communauté active sous le nom de *Manim Community Edition* [3], elle fournit, sous la forme d'une bibliothèque Python, un langage de scène permettant de décrire par le code, avec une grande précision, la géométrie des objets mathématiques et la chronologie des transformations qu'on leur fait subir. Sa force tient à ce qu'elle n'impose pas un répertoire prédéfini d'animations : elle offre au contraire un ensemble de primitives composables, à partir desquelles l'utilisateur construit ses propres figures et ses propres gestes visuels.

1.2 Cadre du projet

Le présent projet s'inscrit dans le module **LU3MA201** (travail d'étude et de recherche) de la licence 3 de mathématiques de Sorbonne Université et a été mené sous la supervision de **M. Frédéric Le Roux**. Il prend pour référence scientifique le *Mémo de topologie* rédigé par F. Le Roux et F. Klopp [1] — document de cours accompagnant le module 3M260 — dont il reprend la progression, les définitions et les exemples canoniques.

L'objectif général est double :

1. concevoir un ensemble d'*animations pédagogiques* couvrant trois grands chapitres du cours, à savoir la **connexité**, la **compacité** et la **complétude** ;
2. organiser ces animations autour d'une *bibliothèque interne* réutilisable, dont la vocation est de factoriser les conventions graphiques, les objets mathématiques récurrents et les schémas d'animation, afin de garantir cohérence visuelle et maintenabilité.

L'enjeu ne se limite donc pas à produire quelques vidéos isolées : il s'agit de bâtir une petite *infrastructure logicielle* dédiée à la visualisation de la topologie métrique, dans laquelle chaque scène peut s'appuyer sur des composants mutualisés (palette chromatique sémantique, primitives d'ensembles topologiques, utilitaires de mise en page, etc.). Cette orientation architecturale, bien qu'ambitieuse pour un projet de licence, nous a semblé conforme à l'esprit du cours lui-même, qui insiste sur la systématité des notions topologiques.

1.3 Périmètre effectif du travail

À l'issue du projet, le dépôt — publié sous le nom `topo-manim` — comporte cinq scènes opérationnelles, chacune associée à une classe *Manim* autonome exécutable en ligne de commande :

- `ConnexeVsArcs`, qui oppose la connexité topologique à la connexité par arcs et prépare le contre-exemple classique $\sin(1/x)$;
- `InvarianceTopologique`, qui illustre l'invariance topologique de la connexité et établit visuellement que $\mathbb{R}$ n'est pas homéomorphe à $\mathbb{R}^2$;
- `ContreExempleSin1x`, qui met en scène l'exemple canonique $\{(x, \sin(1/x)) \mid x > 0\} \cup \{0\} \times [-1, 1]$, espace connexe mais non connexe par arcs ;
- `BorelLebesgue`, qui parcourt la compacité séquentielle, le théorème de Borel–Lebesgue, le théorème de Heine–Borel et le lemme de Lebesgue ;
- `Baire`, qui expose le théorème de Baire, son schéma de preuve par boules emboîtées et l'application classique à la non-dénombrabilité de $\mathbb{R}$.

La bibliothèque interne comporte pour sa part environ mille lignes de code Python réparties entre quatre sous-modules — `config`, `utils`, `objects` et `animations` — auxquels s'ajoutent une cible `Makefile` unifiée et des tests d'import garantissant l'intégrité structurelle du dépôt.

1.4 Plan du rapport

Le rapport est organisé de la manière suivante. Le chapitre 2 décrit l’architecture logicielle du dépôt selon une lecture en couches et justifie les choix de séparation entre code réutilisable et scènes finales. Le chapitre 3 présente le fonctionnement général de *Manim* à partir d’un exemple élémentaire — la visualisation d’un ouvert dans un espace métrique — et dégage l’anatomie commune à toutes les scènes. Le chapitre 4 détaille la bibliothèque interne : palette sémantique, objets visualisables, schémas d’animation. Le chapitre 5 présente les scènes effectivement produites et en commente les choix pédagogiques. Le chapitre 7 propose une étude de cas approfondie du fichier `borel_lebesgue.py`. Le chapitre 6 discute brièvement du flux de travail *Git/GitHub* adopté pour coordonner l’équipe. Enfin, le chapitre 8 tire le bilan du travail effectué et esquisse plusieurs perspectives d’extension.

Architecture logicielle et organisation du dépôt

2.1 Principe directeur : séparer ce qui est réutilisable de ce qui est singulier

La conception du dépôt topo-manim procède d'un principe fondateur qu'il convient d'énoncer d'emblée : *tout ce qui peut être mutualisé doit l'être*, afin qu'une scène particulière ne soit jamais écrite comme un script isolé, dupliquant ses propres constantes, ses propres couleurs et ses propres primitives visuelles. Ce principe répond à une double exigence. Du point de vue logiciel, la mutualisation garantit la maintenabilité du code, évite la dérive conceptuelle entre scènes voisines et rend possible l'extension du projet par de nouvelles animations sans renégocier à chaque fois les conventions graphiques. Du point de vue pédagogique, elle assure au spectateur une cohérence sémantique stable : un ouvert y aura toujours la même couleur, un chemin le même style de trait, une boule le même type d'annotation, si bien que la lecture visuelle d'une scène prépare celle de toutes les autres.

Ce principe directeur se traduit concrètement par une séparation nette entre, d'une part, le répertoire `src/` où se trouvent les briques logicielles réutilisables et, d'autre part, le répertoire `scenes/` où se concentrent les contenus pédagogiques particuliers. Cette distinction, apparemment simple, structure en réalité toute l'organisation du dépôt et en dicte la lecture.

2.2 Lecture architecturale en couches

Pour en rendre compte avec rigueur, il est commode de décomposer le dépôt selon quatre couches fonctionnelles, dont la distinction renvoie non à une partition de dossiers mais à une partition de *responsabilités*.

1. La couche **d'environnement**, constituée de `pyproject.toml`, `Makefile`, `.python-version` et `README.md`, décrit les dépendances, normalise les commandes d'exécution et

documente l'usage du dépôt.

2. La couche **de mutualisation**, matérialisée par le répertoire `src/`, rassemble les composants réutilisables : configuration globale, utilitaires transversaux, objets mathématiques visualisables, schémas d'animation.
3. La couche **de scénarisation**, installée dans `scenes/`, porte les contenus mathématiques effectivement présentés à l'écran, chacun sous la forme d'une classe *Manim* exécutable.
4. La couche **d'artefacts**, composée des répertoires `media/` et `Latex/`, collecte les sorties audiovisuelles engendrées par *Manim* ainsi que la documentation rédigée dont relève le présent rapport.

Cette décomposition a une portée méthodologique. Elle impose que chaque fichier puisse être rattaché à une finalité clairement identifiable — configurer, factoriser, scénariser, produire ou formaliser — et elle rend visible, par là, la qualité architecturale du projet.

TABLE 2.1 – Lecture synoptique de l'architecture du dépôt.

Couche	Contenu principal	Fonction
Environnement	<code>pyproject.toml</code> , <code>Makefile</code> , <code>README.md</code>	Définir les dépendances, standardiser les commandes et documenter l'usage du dépôt.
Mutualisation	<code>src/config.py</code> , <code>src/utils/</code> , <code>src/objects/</code> , <code>src/animations/</code>	Factoriser les conventions graphiques, les objets mathématiques et les mécanismes d'animation réutilisables.
Scénarisation	<code>scenes/01_connexite/</code> , <code>scenes/02_compacite/</code> , <code>scenes/03_completude/</code>	Mettre en scène un contenu mathématique précis sous forme de classe <i>Manim</i> exécutable.
Artefacts	<code>media/</code> , <code>Latex/</code>	Conserver les rendus audiovisuels et la formalisation écrite du projet.

2.3 Couche d'environnement : définir sans ambiguïté le cadre d'exécution

À la racine du dépôt se concentrent les fichiers qui définissent le cadre dans lequel le projet est installé, compilé et exécuté. Leur rôle est moins spectaculaire que celui des scènes mathématiques, mais il est décisif, car c'est d'eux que dépend la reproductibilité du travail sur une machine tierce.

2.3.1 Le fichier `pyproject.toml`

Le fichier `pyproject.toml` déclare le dépôt comme un paquet Python à part entière. Il précise le nom du projet, sa version, la fourchette de versions de Python prises en charge, les dépendances d'exécution — `manim` et `numpy` — et les dépendances de développement — ici, uniquement `pytest`. La minceur délibérée de cette liste traduit un choix de conception : nous avons préféré nous appuyer presque exclusivement sur *Manim* plutôt que d'introduire des dépendances supplémentaires dont la maintenance serait incertaine.

```
1  [build-system]
2  requires = ["setuptools>=68"]
3  build-backend = "setuptools.build_meta"
4
5  [project]
6  name = "topo-manim"
7  version = "0.1.0"
8  description = "Animations Manim pour illustrer le cours de topologie L3"
9  readme = "README.md"
10 requires-python = ">=3.11,<3.14"
11 dependencies = [
12     "manim>=0.18",
13     "numpy",
14 ]
15
16 [dependency-groups]
17 dev = [
18     "pytest",
19 ]
20
21 [tool.setuptools.packages.find]
22 where = ["."]
23
24 [tool.pytest.ini_options]
25 addopts = "-p no:cacheprovider"
26 testpaths = ["tests"]
```

2.3.2 Le Makefile : une interface unifiée de commandes

Le Makefile fournit une couche d'orchestration au-dessus des commandes *Manim*, qui peuvent être longues et sujettes aux erreurs de frappe. Il déclare une cible par scène, puis des cibles agrégées par thème (`connexite`, `compacite`, `completude`) et une cible `all`

rendue par défaut. Une variable `QUALITY` permet de basculer entre rendu rapide (`-ql`, basse qualité) et rendu haute définition (`-qh`), sans duplication de lignes de commande.

```

1 PYTHON = python
2 UV = uv run
3 export PYTHONPATH := $(CURDIR)
4 MANIM = $(UV) manim render
5 QUALITY ?= -ql
6
7 .DEFAULT_GOAL := all
8
9 connexe_vs_arcs:
10     $(MANIM) $(QUALITY) scenes/01_connexite/connexe_vs_arcs.py ConnexeVsArcs
11
12 borel_lebesgue:
13     $(MANIM) $(QUALITY) scenes/02_compacite/borel_lebesgue.py BorelLebesgue
14
15 baire:
16     $(MANIM) $(QUALITY) scenes/03_completude/baire.py Baire
17
18 connexite: connexe_vs_arcs invariance sin1x
19 compacite: borel_lebesgue
20 completude: baire
21
22 all: connexite compacite completude
23
24 hq:
25     $(MAKE) all QUALITY=-qh
26
27 check:
28     $(UV) python -m compileall src scenes tests
29
30 test:
31     $(UV) pytest
32
33 install:
34     uv sync
35
36 clean:
37     $(PYTHON) -c "import shutil; shutil.rmtree('media', ignore_errors=True)"

```

L'intérêt d'un tel Makefile dépasse la simple commodité : il formalise le vocabulaire d'usage du dépôt. Ajouter une nouvelle scène suppose désormais non seulement de

l’écrire, mais aussi de lui associer une cible, ce qui ancre son existence dans la chaîne de production du projet.

2.3.3 Le répertoire `tests/`

Le répertoire `tests/` contient, dans son état courant, un unique fichier `test_scene_imports.py` qui ne cherche pas à tester le contenu mathématique ou visuel des scènes — entreprise difficile à automatiser pour une bibliothèque de rendu — mais qui vérifie deux propriétés structurelles : (i) chaque classe de scène attendue (`ConnexeVsArcs`, `InvarianceTopologique`, `ContreExempleSinix`, `BorelLebesgue`, `Baire`) est bien définie et importable ; (ii) chacun des dix modules de la bibliothèque interne (`src.config`, `src.utils.colors`, etc.) est importable sans erreur. Cette stratégie minimaliste possède une vertu réelle : elle garantit qu’aucune modification n’a corrompu silencieusement l’arborescence, et elle signale immédiatement tout import circulaire ou tout nom manquant.

2.3.4 Les répertoires annexes

Le répertoire `media/` accueille les sorties engendrées par *Manim* — vidéos, images, fichiers $\text{\LaTeX}$ auxiliaires — et n’est donc pas versionné. Le répertoire `Latex/Rapports/` abrite, lui, les sources du présent rapport. Les répertoires techniques (`.venv/`, `topo_manim.egg-info/`, `__pycache__/`) sont nécessaires à l’outillage mais n’appartiennent pas au noyau conceptuel du projet et sont exclus de l’indexation par le fichier `.gitignore`.

2.4 Couche de mutualisation : aperçu du répertoire `src/`

Le répertoire `src/` constitue le cœur logiciel du dépôt. Sa fonction est de factoriser tout ce qui, dans plusieurs scènes, relève d’une convention ou d’un geste récurrent, afin d’éviter la duplication et d’assurer la cohérence graphique. Nous nous contentons ici d’en donner un aperçu synoptique ; le détail des sous-modules et l’analyse de leurs primitives sont développés au chapitre 4.

Le répertoire se subdivise en quatre composantes de responsabilités distinctes :

- `src/config.py` regroupe les paramètres globaux de rendu (résolution, cadence, durées d’animation par défaut) ;
- `src/utils/` rassemble les utilitaires transversaux — palette chromatique sémantique (`colors.py`), primitives de mise en page (`layout.py`) et labels $\text{\LaTeX}$ standardisés (`tex_labels.py`) ;
- `src/objects/` expose les *objects* qui incarnent des objets mathématiques visualisables — espaces métriques, ensembles topologiques, chemins, recouvrements,

dispositifs de présentation ;

- `src/animations/` encapsule des schémas d’animation complets — animations ε - δ , convergence de suites, déformations continues — prêts à être invoqués par les scènes.

Cette organisation traduit une intention : ne pas se limiter à produire des scripts de scène ponctuels mais construire, pas à pas, une petite bibliothèque de gestes visuels réutilisables. Dans l’état courant du dépôt, l’appropriation effective de cette bibliothèque par les scènes est partielle — certaines briques, comme `src/config.py`, `src/utils/colors.py` et `src/objects/presentation.py`, sont déjà mobilisées systématiquement, quand d’autres restent une infrastructure prête pour des développements futurs. Ce point, loin d’être une faiblesse, atteste de la nature *évolutive* du projet.

2.5 Couche de scénarisation : aperçu du répertoire `scenes/`

Le répertoire `scenes/` contient les scènes effectivement rendues par *Manim* et regroupe les fichiers par grands thèmes du cours. Là encore, nous nous limitons ici à une cartographie sommaire, le détail de chaque scène et de son scénario mathématique étant reporté au chapitre 5.

- `scenes/01_connexite/` rassemble trois scènes consacrées à la connexité : `connexe_vs_arcs.py` (classe `ConnexeVsArcs`, environ 1 600 lignes, qui oppose connexité et connexité par arcs), `invariance_topologique.py` (classe `InvarianceTopologique`) et `contre_exemple_sin1x.py` (classe `ContreExempleSin1x`).
- `scenes/02_compacite/` contient le fichier `borel_lebesgue.py` (classe `BorelLebesgue`) qui articule compacité séquentielle, théorème de Borel–Lebesgue, théorème de Heine–Borel et lemme de Lebesgue.
- `scenes/03_completude/` contient le fichier `baire.py` (classe `Baire`) dédié au théorème de Baire et à son application à la non-dénombrabilité de $\mathbb{R}$.

Les fichiers `__init__.py` présents à chaque niveau, quoique pratiquement vides, marquent explicitement ces dossiers comme des *paquets Python* et facilitent les imports croisés.

2.6 Cycle de vie d’une scène : de l’idée mathématique au fichier vidéo

Le fonctionnement du dépôt peut être décrit comme un cycle de transformation d’une idée mathématique en objet audiovisuel. Ce cycle se déroule en cinq temps :

1. une notion du cours est sélectionnée et traduite en objectif pédagogique précis —

- définition, théorème, implication ou contre-exemple;
- 2. cette notion est *scénarisée* dans un fichier de `scenes/`, sous la forme d’une classe héritant de `Scene`, dont la méthode `construct` décrit la chronologie des animations;
- 3. la scène importe, lorsque cela est utile, des composants réutilisables issus de `src/` (constantes, couleurs, primitives);
- 4. le rendu est lancé via *Manim*, directement ou par l’intermédiaire du `Makefile`, typiquement avec la commande `make <nom_de_scene>` ;
- 5. les fichiers visuels produits sont déposés dans `media/`, tandis que les tests d’import garantissent la cohérence structurelle du dépôt.

Ce cycle, apparemment banal, mérite d’être explicité car il fait apparaître une propriété essentielle du projet : le dépôt ne se réduit pas à un assemblage de scripts Python; il matérialise une *chaîne de production pédagogique* dont chaque maillon a sa fonction propre.

2.7 Synthèse cartographique des principaux fichiers

Afin de donner au lecteur une vue d’ensemble, le tableau 2.2 regroupe les principaux fichiers du dépôt selon leur fonction dominante. Cette synthèse n’a pas vocation à se substituer aux chapitres suivants, mais elle permet de fixer, en un seul coup d’œil, la géographie du projet.

TABLE 2.2: Fonction dominante des principaux fichiers du dépôt.

Fichier ou groupe de fichiers	Rôle principal	Usage concret
<code>README.md</code>	Documenter le projet	Présenter les objectifs, l’arborescence et les commandes usuelles.
<code>pyproject.toml</code>	Décrire l’environnement Python	Définir les dépendances, la version du paquet et la configuration de <code>pytest</code> .
<code>Makefile</code>	Orchestrer les tâches	Lancer les rendus, les vérifications et l’installation avec des commandes normalisées.

Fichier ou groupe de fichiers	Rôle principal	Usage concret
src/config.py	Centraliser les paramètres globaux	Uniformiser résolution, fond, cadence et durées d'animation.
src/utils/colors.py	Stabiliser la sémantique visuelle	Associer des couleurs constantes aux concepts topologiques.
src/utils/layout.py	Uniformiser la mise en page	Créer titres, annotations et cadres selon des conventions homogènes.
src/utils/tex_labels.py	Réutiliser des labels mathématiques	Éviter la redéfinition dispersée d'énoncés et de notations fréquentes.
src/objects/*.py	Modéliser des objets mathématiques	Fournir des classes visuelles pour boules, chemins, ensembles et recouvrements.
src/animations/*.py	Encapsuler des schémas d'animation	Préparer des visualisations récurrentes de continuité, convergence ou déformation.
scenes/01_connexite/*.py	Produire les scènes sur la connexité	Mettre en images définitions, invariance et contre-exemples liés à la connexité.
scenes/02_compacite/*.py	Produire les scènes sur la compacité	Illustrer la compacité séquentielle et le théorème de Borel–Lebesgue.
scenes/03_completude/*.py	Produire les scènes sur la complétude	Visualiser le théorème de Baire et ses conséquences.
tests/test_scene_import.py	Vérifier la cohérence structurelle	S'assurer que les modules et les classes attendues s'importent correctement.
media/	Conserver les sorties générées	Archiver vidéos, images et fichiers intermédiaires produits par <i>Manim</i> .
Latex/Rapports/	Formaliser le projet par écrit	Produire une documentation académique parallèle au code.

2.8 Appréciation critique de l'état actuel

La structure du dépôt est, dans son principe, saine et méthodiquement pensée. Elle distingue nettement le code réutilisable du code de production, centralise les conventions graphiques dans un petit nombre de points d'autorité — la palette de `colors.py`, la configuration de `config.py`, le `Makefile` — et adopte une nomenclature lisible, organisée par thèmes mathématiques conformes à la progression du cours. Les responsabilités de chaque couche sont clairement séparées : l'environnement déclare les dépendances, la bibliothèque interne factorise les primitives, la scénarisation porte le contenu pédagogique, et la couche d'artefacts collecte les sorties engendrées.

Pour autant, le degré de *factorisation effective* n'est pas uniforme sur l'ensemble du dépôt. En pratique, les scènes s'appuient déjà largement sur `src/config.py`, `src/utils/colors.py` et `src/objects/presentation.py` ; en revanche, plusieurs classes plus abstraites du sous-module `src/objects/` (notamment `TopologicalSet` et `MetricSpace`) ainsi que la quasi-totalité de `src/animations/` ne sont pas encore mobilisées de manière systématique dans les scènes finales. Cette observation ne révèle pas une faiblesse conceptuelle : elle témoigne simplement du caractère évolutif d'un projet qui n'a pas encore atteint son régime de croisière. L'architecture modulaire a été posée ; une part de cette bibliothèque est déjà prête pour des extensions futures et sera naturellement investie par de nouvelles scènes.

En ce sens, le dépôt présente une *double nature* qu'il convient d'assumer. D'une part, c'est un outil de production d'animations mathématiques déjà opérationnel, dont les cinq scènes actuelles couvrent un spectre représentatif du cours de topologie. D'autre part, il tend vers une bibliothèque plus générale de visualisation topologique, qui pourrait être enrichie à mesure que de nouvelles scènes — sur les espaces normés, la convexité, les topologies produites, etc. — viendraient s'y ajouter. Cette tension entre un état opérationnel et une intention plus ambitieuse est, nous semble-t-il, le signe d'une maturité logicielle en construction : la cohérence conceptuelle est en place ; l'alignement entre infrastructure et usage systématique reste à parachever.

Anatomie d'une scène *Manim* : principes et exemple fondateur

3.1 Manim comme langage descriptif de scènes

Avant de détailler la bibliothèque interne et les scènes produites, il convient d'exposer le modèle de programmation sur lequel repose *Manim*, car il conditionne la manière dont nous avons pensé l'architecture du dépôt. *Manim* n'est pas un outil de dessin : c'est un *langage descriptif de scènes*. Une scène y est définie non par la suite des pixels à afficher, mais par une classe Python dont la structure encode successivement les objets mathématiques à représenter, leurs propriétés visuelles et la chronologie des transformations qu'on leur fait subir.

Concrètement, toute scène est une classe qui hérite de la classe mère `Scene` et qui doit implémenter une méthode `construct` sans argument. C'est à l'intérieur de cette méthode que l'on crée les objets graphiques — appelés *mobjects* dans le vocabulaire *Manim* — et que l'on ordonne les animations successives par appel à `self.play(...)` et `self.wait(...)`. Le squelette minimal d'une scène est donc le suivant.

```
1 from manim import Scene
2
3 class MaScene(Scene):
4     def construct(self) -> None:
5         # 1. Définition des objets graphiques (mobjects)
6         # 2. Orchestration des animations successives
7         pass
```

Cette architecture, qui peut paraître élémentaire, présente plusieurs vertus méthodologiques. En premier lieu, elle sépare rigoureusement *ce qui est montré* (les objets créés) et *comment cela est montré* (la séquence d'animations), ce qui facilite la relecture du code et son évolution. En second lieu, elle se prête à une subdivision naturelle en sections : rien n'empêche d'appeler, depuis `construct`, une suite de méthodes auxiliaires dédiées chacune à une idée mathématique particulière. C'est d'ailleurs la convention que nous

avons systématiquement adoptée dans les scènes du projet, comme nous le verrons au chapitre 5.

3.2 Un exemple fondateur : visualisation d'un ensemble ouvert

Pour fixer les idées, nous détaillons dans cette section un exemple minimal qui fut également notre première scène exploratoire. Il s'agit d'illustrer visuellement la définition d'un ensemble ouvert dans un espace métrique, définition que nous rappelons ci-dessous.

Définition

Soit (X, d) un espace métrique. Une partie $O \subset X$ est dite **ouverte** dans X si, pour tout point $x \in O$, il existe un réel $\varepsilon > 0$ tel que la boule ouverte $B(x, \varepsilon)$ soit entièrement contenue dans O . Autrement dit :

$$\forall x \in O, \exists \varepsilon > 0, B(x, \varepsilon) \subset O.$$

Du point de vue visuel, cette définition appelle trois éléments : un ensemble modélisé par un disque, un point intérieur clairement marqué, et une boule ouverte centrée en ce point, strictement incluse dans le disque. Pour que la boule soit, par construction, contenue dans l'ensemble, il suffit d'en fixer le rayon :

$$\varepsilon = r - \|x - c\|,$$

où c et r désignent respectivement le centre et le rayon du disque, et où x est le point intérieur choisi. Cette formule garantit que la boule tangentielle à la frontière reste à l'intérieur du disque tout en n'étant pas triviale.

Remarque

Une telle illustration ne prétend évidemment pas démontrer la définition : elle fournit, dans un cas particulier en dimension 2, un *témoin visuel* qui rend la propriété plus immédiatement saisissable. L'intention pédagogique est de transformer un énoncé quantifié — qui peut paraître abstrait — en une image dans laquelle l'existence d'un ε adéquat devient évidente.

Le code ci-dessous met en œuvre cette idée. Il illustre, sur un cas minimal, la totalité de l'anatomie d'une scène *Manim* : importations, classe héritant de `Scene`, méthode `construct`, création de *objects*, orchestration des animations.

```

1  from manim import (
2      Scene, Circle, Dot, Line,
3      FadeIn, GrowFromCenter, GrowFromPoint,
4      BLUE, YELLOW, GREEN, RED, PI,
5  )
6  import numpy as np
7
8
9  class DefOuvert(Scene):
10     """Illustration visuelle de la définition d'un ouvert dans  $\mathbb{R}^2$ ."""
11
12     def construct(self) -> None:
13         # --- 1. Paramètres géométriques de la situation -----
14         centre_ensemble = np.array([0.0, 0.0, 0.0])
15         rayon_ensemble = 1.5
16         position_point = np.array([-0.15, -0.20, 0.0])
17
18         # Rayon maximal d'une boule ouverte centrée en x, incluse dans D :
19         # epsilon = r - ||x - c||.
20         epsilon = rayon_ensemble - np.linalg.norm(
21             position_point - centre_ensemble
22         )
23
24         # --- 2. Création des mobjects -----
25         ensemble = Circle(
26             radius=rayon_ensemble,
27             color=BLUE,
28             fill_color=BLUE,
29             fill_opacity=0.25,
30             stroke_width=0,
31         ).move_to(centre_ensemble)
32
33         point = Dot(point=position_point, color=YELLOW, radius=0.06)
34
35         boule = Circle(
36             radius=epsilon,
37             color=GREEN,
38             fill_color=GREEN,
39             fill_opacity=0.50,
40             stroke_width=0,
41         ).move_to(position_point)
42

```

```

43     rayon = Line(
44         point.get_center(),
45         boule.point_at_angle(7 * PI / 8),
46         color=RED,
47     )
48
49     # --- 3. Orchestration des animations -----
50     self.play(FadeIn(ensemble), FadeIn(point))
51     self.wait()
52     self.play(
53         GrowFromCenter(boule),
54         GrowFromPoint(rayon, rayon.get_start()),
55     )
56     self.wait()

```

3.3 Lecture analytique de l'animation

Le code ci-dessus, délibérément court, se prête à une analyse point par point qui fait ressortir les règles générales que nous avons ensuite systématisées dans les scènes du projet.

La première étape — `FadeIn(ensemble), FadeIn(point)` — fait apparaître simultanément l'ensemble et le point choisi. Une courte pause, réalisée par `self.wait()`, laisse au spectateur le temps d'identifier visuellement les éléments déjà présents à l'écran avant qu'une nouvelle information n'y soit ajoutée. Cette discipline du silence — que l'on pourrait appeler un *temps de lecture* — est essentielle dans les animations mathématiques : le spectateur a besoin de moments où rien ne se passe pour intégrer ce qui vient de se passer.

La seconde étape fait croître la boule ouverte à partir de son centre grâce à `GrowFromCenter`, cependant que le segment représentant le rayon ε croît à partir du point x grâce à `GrowFromPoint`. Le choix de ces deux animations n'est pas neutre : en faisant naître la boule depuis son centre et le rayon depuis le point intérieur, nous rendons sensible visuellement l'idée que la boule est *engendrée* à partir du point considéré, et non arbitrairement posée sur la figure. L'animation exprime ainsi, dans son geste même, la structure logique de la définition : à chaque point, on associe une boule.

Enfin, sur le plan pédagogique, cette scène minimale accomplit trois fonctions. Elle matérialise la présence d'un point intérieur, elle rend tangible l'existence concrète d'une boule incluse dans l'ensemble, et elle prépare des animations ultérieures portant sur d'autres notions topologiques étroitement apparentées — fermés, intérieur, adhérence, frontière — qui se déduisent toutes, d'une manière ou d'une autre, de la notion d'ouvert.

3.4 Ce que l'exemple nous a appris

Cet exemple fondateur nous a conduits à dégager quatre enseignements méthodologiques qui ont ensuite gouverné l'ensemble du projet :

1. une scène *Manim* se lit comme une *phrase visuelle* dont la syntaxe est la chronologie des animations ; il faut donc la concevoir comme un discours, avec un rythme, des pauses et des liaisons logiques ;
2. le code d'une scène gagne en lisibilité lorsqu'il sépare clairement la *définition des paramètres*, la *création des mobjects* et l'*orchestration des animations* ; nous avons retenu cette tripartition comme canon d'écriture ;
3. les constantes récurrentes — couleurs, durées d'attente, tailles — doivent être externalisées, faute de quoi chaque nouvelle scène dérive insensiblement de la précédente ; c'est ce constat qui a motivé la création du module `src/config.py` et du module `src/utils/colors.py` ;
4. enfin, certains gestes visuels reviennent d'une scène à l'autre — apparition d'une boule ouverte, mise en évidence d'un point, animation ε - δ — et gagnent à être encapsulés dans des primitives réutilisables ; c'est l'origine des modules `src/objects/` et `src/animations/`.

Autrement dit, la prise en main de *Manim* n'a pas seulement consisté à apprendre le vocabulaire d'une bibliothèque : elle nous a conduits à poser les fondations architecturales du dépôt. Le chapitre suivant présente en détail ces fondations.

La bibliothèque interne : une boîte à outils pour la topologie visuelle

4.1 Configuration globale : le module `src/config.py`

Toute scène *Manim* dépend de paramètres globaux — résolution, cadence d’images, couleur de fond, répertoire de sortie — qui doivent rester homogènes d’une animation à l’autre, sous peine de donner au résultat final un aspect hétéroclite. Pour garantir cette homogénéité, nous avons centralisé ces paramètres dans un unique fichier `src/config.py` que toutes les scènes importent et auquel nul code ne déroge. Ce fichier définit d’une part les paramètres de rendu de *Manim* et d’autre part trois constantes de temporisation partagées — `DEFAULT_ANIM_DURATION`, `DEFAULT_WAIT` et `SHORT_WAIT` — qui règlent le rythme narratif des animations.

Le rôle de ce module est moins technique que méthodologique : en imposant un point d’autorité unique pour les paramètres globaux, il empêche la dérive silencieuse qui se produit inmanquablement lorsqu’une scène redéfinit localement une constante que sa voisine ne connaît pas. En pratique, toutes nos scènes importent au minimum `DEFAULT_WAIT` et `SHORT_WAIT` depuis ce module, ce qui garantit que les pauses narratives — ces moments de silence qui laissent au spectateur le temps d’intégrer une nouvelle information — ont partout la même durée.

4.2 Palette chromatique sémantique : `src/utils/colors.py`

La couleur, dans une animation mathématique, n’est pas un élément décoratif mais un véhicule sémantique à part entière. Notre projet repose sur le principe suivant : *à chaque notion mathématique doit correspondre une couleur stable*, et réciproquement. C’est ce principe que matérialise le module `src/utils/colors.py`, qui définit l’ensemble des couleurs utilisées dans le dépôt sous la forme d’objets `ManimColor`.

```

1  """Palette de couleurs cohérente pour le projet topo-manim."""
2
3  from manim import ManimColor
4
5  # --- Espaces et ensembles ---
6  SPACE_COLOR = ManimColor("#2B2D42")      # fond d'espace métrique
7  OPEN_SET_COLOR = ManimColor("#06D6A0")    # ouverts
8  CLOSED_SET_COLOR = ManimColor("#EF476F")  # fermés
9  INTERIOR_COLOR = ManimColor("#118AB2")    # intérieur
10 BOUNDARY_COLOR = ManimColor("#FFD166")    # frontière
11 CLOSURE_COLOR = ManimColor("#073B4C")     # adhérence
12
13 # --- Chemins et arcs ---
14 PATH_COLOR = ManimColor("#F72585")
15 ARC_COLOR = ManimColor("#7209B7")
16
17 # --- Recouvrements ---
18 COVER_COLORS = [
19     ManimColor("#4361EE"),
20     ManimColor("#4CC9F0"),
21     ManimColor("#F72585"),
22     ManimColor("#7209B7"),
23     ManimColor("#3A0CA3"),
24 ]
25
26 # --- Epsilon-delta -> Continuité ---
27 EPSILON_COLOR = ManimColor("#06D6A0")
28 DELTA_COLOR = ManimColor("#FFD166")
29
30 # --- Général ---
31 HIGHLIGHT_COLOR = ManimColor("#F77F00")
32 DIM_COLOR = ManimColor("#8D99AE")
33 TEXT_COLOR = ManimColor("#EDF2F4")

```

On remarquera la structuration explicite du fichier en catégories sémantiques — espaces et ensembles, chemins et arcs, recouvrements, ε - δ , général — qui préfigure la taxonomie des objets mathématiques manipulés par les scènes. Cette classification n'est pas gratuite : elle facilite la relecture, guide le choix de la couleur lors de l'écriture d'une nouvelle scène et formalise une véritable *grammaire chromatique* du projet. Par exemple, `OPEN_SET_COLOR` (vert) et `CLOSED_SET_COLOR` (rose) définissent, d'un bout à l'autre du dépôt, le code de lecture pour distinguer les ouverts des fermés. De même, la liste `COVER_COLORS` propose une séquence de cinq teintes destinée à colorier les ouverts d'un

recouvrement et donnant lieu, lorsqu'elle est cyclée, à des visualisations lisibles même pour une famille de taille supérieure.

4.3 Utilitaires de mise en page : src/utils/layout.py

Complément fonctionnel du module de couleurs, le fichier src/utils/layout.py rassemble quelques fonctions simples, mais fréquemment invoquées, qui assurent l'homogénéité de la composition spatiale des scènes. On y trouve notamment une fonction créant un titre placé en haut de l'écran, une fonction traçant un cadre englobant autour d'un *mobject* et une fonction ajoutant une annotation L^AT_EX à côté d'un objet. Le code ci-dessous en reproduit un extrait représentatif :

```

1  """Utilitaires de positionnement, légendes et annotations."""
2
3  from manim import (
4      VGroup, Text, MathTex, Rectangle, DOWN, RIGHT, LEFT, UP,
5      ORIGIN, config, ManimColor,
6  )
7
8
9  def title_text(text: str, font_size: int = 40, **kwargs) -> Text:
10     """Crée un titre positionné en haut de l'écran."""
11     t = Text(text, font_size=font_size, **kwargs)
12     t.to_edge(UP, buff=0.5)
13     return t
14
15
16 def annotate(mobject, label_text: str, direction=DOWN, font_size: int = 22):
17     """Ajoute une annotation LaTeX sous un mobject."""
18     label = MathTex(label_text, font_size=font_size)
19     label.next_to(mobject, direction, buff=0.25)
20     return label
21
22
23 def bounding_box(mobject, buff: float = 0.2, **kwargs) -> Rectangle:
24     """Rectangle englobant un mobject."""
25     rect = Rectangle(
26         width=mobject.width + 2 * buff,
27         height=mobject.height + 2 * buff,
28         **kwargs,
29     )

```

```

30     rect.move_to(mobject)
31     return rect

```

Chacune de ces fonctions est délibérément courte : son unique rôle est de *nommer* et d'*uniformiser* un geste de mise en page que plusieurs scènes seraient autrement tentées de réinventer. On en mesure la valeur dès qu'il faut modifier globalement la position d'un titre ou la marge d'une annotation : une seule ligne à ajuster dans `layout.py` suffit à répercuter le changement à travers tout le dépôt.

4.4 Labels L^AT_EX standardisés : `src/utils/tex_labels.py`

Le troisième utilitaire, `src/utils/tex_labels.py`, regroupe des expressions L^AT_EX standardisées correspondant à des notions récurrentes du cours : quantificateurs, boules ouvertes, opérateurs d'adhérence, d'intérieur et de frontière, définitions formelles de la connexité, de la connexité par arcs et du théorème de Borel–Lebesgue. Chaque label y est exposé sous la forme d'une fonction renvoyant un `MathTex` déjà paramétré. Cette centralisation empêche la dispersion de variantes typographiques dans le dépôt : une boule ouverte y sera toujours affichée comme $B(x, r)$, et non parfois $B_r(x)$, parfois $\mathcal{B}(x; r)$.

4.5 Objets mathématiques visualisables : `src/objects/`

Le sous-répertoire `src/objects/` regroupe les classes qui incarnent les objets mathématiques du cours sous la forme de *mobjects* *Manim*. Cinq classes principales y sont définies, chacune correspondant à une notion topologique élémentaire.

- `MetricSpace` (dans `metric_space.py`) représente une portion de plan $\mathbb{R}^2$ munie éventuellement d'axes ; elle expose une méthode `coords_to_point` qui convertit des coordonnées métriques en coordonnées d'écran, et une méthode `add_ball` qui y ajoute une boule ouverte.
- `OpenBall` (dans `metric_space.py`) représente une boule ouverte $B(c, r)$, comprenant son disque, son centre matérialisé par un point et, facultativement, une étiquette L^AT_EX.
- `TopologicalSet` (dans `topological_set.py`) représente un sous-ensemble topologique générique ; un paramètre booléen `is_open` détermine si la frontière est tracée en pointillés (ouvert) ou en trait plein (fermé). La classe expose également les méthodes `get_interior` et `get_boundary_highlight`.
- `Path` et `Arc` (dans `paths.py`) représentent respectivement un chemin continu $\gamma : [0, 1] \rightarrow X$ et un arc ; le fichier expose en outre une fonction `concatenate_paths`

qui réalise la concaténation $\gamma_1 \star \gamma_2$.

- `Covering` (dans `coverings.py`) représente un recouvrement ouvert défini par une liste de centres et de rayons ; la classe expose les méthodes `extract_subcover` et `highlight_subcover`, directement motivées par la mise en scène du théorème de Borel–Lebesgue.

Le code qui suit reproduit l’implémentation de la fonction `concatenate_paths`, petit mais conceptuellement éloquent : elle traduit en Python la définition classique de la concaténation de deux chemins, opération essentielle pour établir la transitivité de la connexité par arcs.

```

1 def concatenate_paths(
2     gamma1: Callable, gamma2: Callable
3 ) -> Callable[[float], np.ndarray]:
4     """Concaténation gamma1 * gamma2 : [0,1] -> X."""
5     def concatenated(t: float) -> np.ndarray:
6         if t <= 0.5:
7             return gamma1(2 * t)
8         return gamma2(2 * t - 1)
9     return concatenated

```

À côté de ces classes spécifiquement mathématiques, le fichier `src/objects/presentation.py` rassemble des primitives de présentation plus génériques — halos lumineux autour d’un point, panneaux pédagogiques à titre coloré — qui ne modélisent pas des objets topologiques mais des *dispositifs de mise en scène*. Ces primitives jouent un rôle décisif dans les scènes les plus travaillées visuellement, comme `ContreExempleSin1x`, où l’argument mathématique est accompagné d’un habillage narratif fort.

4.6 Schémas d’animation réutilisables : `src/animations/`

Le sous-répertoire `src/animations/` contient des classes qui encapsulent non plus des objets, mais des *schémas d’animation complets*. Trois modules y sont actuellement définis :

- `continuity.py` définit la classe `EpsilonDeltaAnimation`, qui anime la construction classique d’un argument de continuité en faisant décroître simultanément un disque de rayon ε dans l’espace image et un disque de rayon δ dans l’espace source ; ainsi que la classe `PreimageOpenSet`, qui illustre le fait que l’image réciproque d’un ouvert par une application continue est un ouvert.
- `convergence.py` définit les classes `SequenceConvergence` et `CauchySequence`, qui visualisent respectivement la convergence d’une suite et le caractère de Cauchy

d’une suite en représentant la *queue* où tous les termes sont confinés.

- `deformations.py` définit les classes `ContinuousDeformation` et `HomeomorphismDemo`, qui animent des déformations continues entre deux formes (typiquement un cercle et un carré) pour illustrer visuellement un homéomorphisme.

Comme cela a été remarqué au chapitre 2, ces classes n’ont pas encore toutes été invoquées par les scènes finales. Elles constituent néanmoins l’ossature d’une bibliothèque de gestes visuels réutilisables qui pourra être mobilisée dans les extensions ultérieures du dépôt.

4.7 Remarque méthodologique

Les modules décrits ci-dessus n’ont pas été conçus d’un jet ni d’avance : ils ont émergé au fil de l’écriture des scènes, à mesure que des motifs récurrents apparaissaient et appelaient une factorisation. C’est en réalité la trajectoire habituelle d’un projet logiciel bien conduit : les abstractions que l’on crée le plus légitimement sont celles que le besoin a d’abord rendues évidentes. La bibliothèque interne de `topo-manim` est ainsi le résultat d’un va-et-vient constant entre le concret — écrire une scène — et l’abstrait — en extraire ce qui peut servir à d’autres.

Les scènes produites : contenus mathématiques et choix pédagogiques

5.1 Vue d'ensemble

Le dépôt comprend cinq scènes principales, réparties sur les trois grands thèmes du cours. Le tableau 5.1 en donne un aperçu synoptique.

TABLE 5.1 – Aperçu des scènes produites.

Classe	Thème	Contenu mathématique
<code>ConnexeVsArcs</code>	Connexité	Opposition entre connexité et connexité par arcs ; implication et contre-exemple.
<code>InvarianceTopologique</code>	Connexité	Invariance de la connexité par homéomorphisme ; $\mathbb{R} \not\cong \mathbb{R}^2$.
<code>ContreExempleSin1x</code>	Connexité	Contre-exemple $\sin(1/x)$: espace connexe, non connexe par arcs.
<code>BorelLebesgue</code>	Compacité	Compacité séquentielle, Borel–Lebesgue, Heine–Borel, lemme de Lebesgue.
<code>Baire</code>	Complétude	Théorème de Baire ; preuve par boules emboîtées ; $\mathbb{R}$ non dénombrable.

Chacune de ces scènes est organisée selon un même patron : une méthode `construct` minimale qui se contente d'appeler, dans l'ordre voulu, une suite de méthodes `section_...` dédiées chacune à une idée mathématique précise. Cette structuration, qui s'est imposée à l'usage, présente un triple avantage. Elle permet de faire correspondre un bloc de code à une unité pédagogique ; elle facilite la relecture et la révision ; elle rend possible la réorganisation du scénario en simple réordonnancement d'appels de méthodes.

5.2 La scène ConnexeVsArcs : opposer deux intuitions

Le fichier `scenes/01_connexite/connexe_vs_arcs.py` est la scène la plus ambitieuse du projet, avec environ 1 600 lignes de code. Elle suit fidèlement le chapitre IV du *Mémo de topologie* [1] et expose, en six sections successives, la différence entre la connexité topologique et la connexité par arcs.

```

1  from src.config import DEFAULT_WAIT
2  from src.objects.presentation import make_glow
3  from src.utils.colors import (
4      DIM_COLOR, HIGHLIGHT_COLOR, OPEN_SET_COLOR, PATH_COLOR, TEXT_COLOR,
5  )
6
7
8  class ConnexeVsArcs(Scene):
9      """Connexité vs connexité par arcs (chapitre IV du cours)."""
10
11     def construct(self) -> None:
12         self.section_ouverture()
13         self.section_connexite_par_arcs()
14         self.section_connexite()
15         self.section_caracterisation()
16         self.section_implication()
17         self.section_reciproque()
18
19     def section_ouverture(self) -> None:
20         """Pose la question intuitive et révèle le titre."""
21         question = Text(
22             "Que veut dire qu'un espace est « d'un seul tenant » ?",
23             font_size=32,
24             color=TEXT_COLOR,
25         )
26         self.play(Write(question), run_time=2)
27         self.wait(1.2)

```

La progression adoptée mérite d'être détaillée, car elle exemplifie la démarche pédagogique de l'ensemble du projet.

1. **Ouverture.** La scène s'ouvre sur une question intuitive : « Que veut dire qu'un espace est « d'un seul tenant » ? ». Deux réponses candidates sont proposées au spectateur : (i) on peut joindre deux points par un chemin continu ; (ii) on ne peut pas le découper en deux ouverts disjoints. Cette ouverture est capitale car elle

pose d'emblée la question qui va structurer l'ensemble du scénario : ces deux réponses sont-elles équivalentes ?

2. **Connexité par arcs.** La première notion est définie : un espace X est connexe par arcs si, pour tous $a, b \in X$, il existe un chemin continu $\gamma : [0, 1] \rightarrow X$ tel que $\gamma(0) = a$ et $\gamma(1) = b$. Cette définition est immédiatement illustrée par l'animation d'un chemin continu joignant deux points d'un disque.
3. **Connexité topologique.** La seconde notion est ensuite introduite : un espace X est connexe s'il n'existe pas de partition $X = O_1 \sqcup O_2$ en deux ouverts non vides disjoints. L'animation visualise cette définition en montrant une tentative de découpage qui, nécessairement, laisse les deux ouverts se « toucher ».
4. **Caractérisation.** La scène énonce ensuite une caractérisation classique : X est connexe si et seulement si toute application continue $X \rightarrow \{0, 1\}$ est constante. Cette reformulation prépare le terrain à l'argument d'implication.
5. **Implication.** Il est alors démontré — plus exactement, visuellement argumenté — que la connexité par arcs implique la connexité. L'argument repose sur la continuité de $\gamma : [0, 1] \rightarrow X$ et sur la connexité de $[0, 1]$.
6. **Réciproque.** La scène se clôt sur la question de la réciproque. Un contre-exemple est annoncé — celui de l'ensemble $\sin(1/x)$ — dont l'exposition détaillée est renvoyée à la scène `ContreExempleSin1x`.

La valeur de cette scène tient à ce qu'elle n'expose pas seulement deux définitions : elle les *oppose*, elle les *relie* et elle prépare leur *dissociation* par un contre-exemple. Le scénario visuel y épouse, pas à pas, le mouvement du raisonnement mathématique.

5.3 La scène InvarianceTopologique

Plus concise, la scène `InvarianceTopologique` illustre le fait que la connexité est une propriété *topologique*, c'est-à-dire préservée par tout homéomorphisme. L'application classique proposée consiste à en déduire que $\mathbb{R}$ n'est pas homéomorphe à $\mathbb{R}^2$: en effet, si $\varphi : \mathbb{R} \rightarrow \mathbb{R}^2$ était un homéomorphisme, alors l'image $\varphi(\mathbb{R} \setminus \{0\})$ serait un ensemble non connexe homéomorphe à $\mathbb{R}^2 \setminus \{\varphi(0)\}$, lequel est pourtant connexe, ce qui est contradictoire. La scène visualise cet argument en représentant le « retrait d'un point » dans les deux espaces, faisant immédiatement apparaître la différence de comportement.

5.4 La scène ContreExempleSin1x

La scène `ContreExempleSin1x` est, du point de vue mathématique, celle qui porte le contenu le plus délicat. Il s'agit d'exposer l'exemple canonique

$$S = \{(x, \sin(1/x)) \mid x > 0\} \cup (\{0\} \times [-1, 1]),$$

qui est un espace connexe mais non connexe par arcs. Le spectateur y est amené à comprendre pourquoi, bien que tout voisinage du segment $\{0\} \times [-1, 1]$ contienne des points du graphe, aucun chemin continu ne peut aller de l'un à l'autre. L'oscillation indéfinie de $\sin(1/x)$ au voisinage de 0 y est rendue visuellement sensible. Cette scène mobilise intensivement les primitives de `src/objects/presentation.py` : panneaux pédagogiques, halos, annotations progressives, qui permettent d'accompagner le spectateur tout au long d'un raisonnement non trivial.

5.5 La scène BorelLebesgue

La scène `BorelLebesgue`, longue d'environ 280 lignes, est analysée en profondeur au chapitre 7 auquel nous renvoyons le lecteur. Nous nous bornons ici à en rappeler l'architecture : six sections successives — titre, compacité séquentielle, recouvrement, segment compact, exemple non compact, lemme de Lebesgue — qui articulent, dans un même scénario, cinq formulations complémentaires de la compacité.

5.6 La scène Baire

Le fichier `scenes/03_completude/baire.py` implémente la scène `Baire`, consacrée au théorème de Baire. Le scénario s'ouvre sur l'énoncé du théorème — dans un espace métrique complet, toute intersection dénombrable d'ouverts denses est elle-même dense — puis rappelle la définition d'un ouvert dense, avant de visualiser le schéma de la preuve classique par boules emboîtées.

```

1 steps = VGroup(
2     MathTex(
3         r"1.\ \text{Comme } U_n \text{ est dense, } "
4         r"B_n \cap U_n \neq \varnothing.",
5         font_size=20,
6     ),
7     MathTex(
8         r"2.\ \text{On choisit } B_{n+1} \subset B_n \cap U_n "
```

```

9      r"\text{ plus petite.}]",
10      font_size=20,
11  ),
12  MathTex(
13      r"3.\ \text{Les centres forment une suite de Cauchy.}",
14      font_size=20,
15  ),
16  MathTex(
17      r"4.\ \text{La complétude donne une limite } "
18      r"x \in \bigcap_n B_n \subset \bigcap_n U_n.",
19      font_size=20,
20      color=OPEN_SET_COLOR,
21  ),
22 ).arrange(DOWN, buff=0.18, aligned_edge=LEFT).to_edge(DOWN, buff=0.2)

```

Les quatre étapes affichées à l'écran correspondent exactement au schéma de preuve du cours : densité de U_n , extraction d'une boule plus petite, caractère de Cauchy de la suite des centres, et usage de la complétude pour conclure. La scène se clôt par une application remarquable : le théorème de Baire permet de démontrer, de manière non constructive mais concise, que $\mathbb{R}$ n'est pas dénombrable. La force de cette application réside dans le renversement logique qu'elle opère : la complétude, propriété « positive », entraîne une propriété « négative » — l'impossibilité d'énumérer $\mathbb{R}$.

5.7 Cohérence pédagogique de l'ensemble

Prises ensemble, ces cinq scènes forment un parcours cohérent qui parcourt trois des grands théorèmes structurant le cours de topologie métrique. La progression y est à la fois verticale — chaque scène s'enrichit de celles qui l'ont précédée — et horizontale — les thèmes (connexité, compacité, complétude) sont relativement autonomes et peuvent être abordés dans n'importe quel ordre. La palette chromatique commune, le rythme narratif imposé par les constantes de temporisation partagées et l'usage récurrent des primitives de présentation garantissent que ce parcours apparaît, au spectateur, comme un ensemble cohérent et non comme une simple juxtaposition.

Utilisation de GitHub et flux de travail Git

6.1 Rôle de Git et de GitHub dans un projet collaboratif

Dans un projet comme *topo-manim*, la production ne se limite pas à l'écriture de fichiers Python : elle implique également la conservation de l'historique du travail, la comparaison entre versions successives, la coordination entre plusieurs contributeurs et la publication d'un état stable du dépôt. C'est précisément le rôle du couple *Git* / *GitHub*, dont l'emploi s'est imposé dès les premières semaines du projet.

Git est un système de gestion de versions distribué : il enregistre localement, sous forme d'instantanés (*commits*), les modifications apportées à un projet, il autorise le retour à un état antérieur, la création de branches de travail indépendantes et la fusion d'évolutions concurrentes. *GitHub*, de son côté, est une plateforme d'hébergement distant de dépôts *Git* qui ajoute à ces fonctionnalités élémentaires un ensemble d'outils de collaboration : dépôt partagé, interface de revue de code, comparaison visuelle des modifications, ouverture et discussion de *pull requests*, gestion d'issues.

Dans le cadre d'un projet mené à quatre contributeurs, ce dispositif n'est pas un luxe : il est une condition même de possibilité du travail collectif. Il répond à quatre exigences essentielles.

1. Il garantit la **traçabilité** du développement des scènes et des modules réutilisables : à tout instant, il est possible de savoir qui a modifié quoi, quand et pourquoi.
2. Il rend possible un **travail parallèle** non destructif, en autorisant plusieurs membres à intervenir simultanément sur des fichiers distincts sans risque d'écrasement mutuel.
3. Il fournit une **sauvegarde distante** du dépôt, complémentaire du travail local et résistante aux défaillances matérielles individuelles.
4. Il matérialise une **discipline de publication** : seuls les états jugés cohérents sont publiés sur la branche principale, tandis que les expérimentations restent confinées à des branches dédiées.

6.2 Le cycle élémentaire d'utilisation

L'usage de *Git* repose sur un cycle simple, mais dont le respect conditionne la qualité de l'historique produit. Ce cycle peut être résumé en cinq opérations canoniques :

1. mettre à jour localement l'état du dépôt distant (`git pull`);
2. modifier, dans le dépôt local, les fichiers utiles au travail en cours;
3. placer dans l'*index* les modifications retenues pour le prochain *commit* (`git add`);
4. enregistrer un instantané cohérent par un *commit* (`git commit`);
5. publier cet instantané sur le dépôt distant (`git push`).

Derrière cette apparente simplicité se cache en réalité une véritable discipline intellectuelle : chaque *commit* doit correspondre à une unité de travail cohérente et identifiable, dont le message d'accompagnement formule avec précision ce qui a été fait et, idéalement, pourquoi.

6.3 Commandes fondamentales : une brève anthologie

Nous rappelons ci-après les commandes qui ont structuré notre pratique quotidienne de *Git* pendant le projet. Chacune est reproduite dans un environnement terminal afin de refléter au plus près l'expérience réelle de leur usage.

6.3.1 Récupération ou initialisation du dépôt

Le dépôt étant hébergé sur *GitHub*, chaque contributeur commence par en obtenir une copie locale au moyen de la commande `git clone` :

```
$ Terminal
$ git clone https://github.com/<organisation>/topo-manim.git
$ cd topo-manim
```

Dans le cas symétrique où le projet est d'abord créé localement puis poussé vers un dépôt distant vierge, on procède plutôt par `git init` suivi de `git remote add`.

```
$ Terminal
$ git init
$ git remote add origin https://github.com/<organisation>/topo-manim.git
```

Dans les deux cas, la finalité est la même : disposer d'un dépôt local relié à une référence distante partagée qui sert de point de synchronisation entre les contributeurs.

6.3.2 Inspection de l'état local

La commande la plus fréquemment invoquée au quotidien est `git status`. Elle dresse la liste des fichiers modifiés, indique ceux qui sont déjà placés dans l'index pour le prochain *commit* et signale ceux qui restent non suivis.

```
$ Terminal  
$ git status
```

Il est de bonne pratique d'exécuter cette commande avant et après chaque série de modifications significatives, ne serait-ce que pour s'assurer qu'aucune modification involontaire ne se glisse dans le prochain *commit*.

6.3.3 Enregistrement d'une modification

L'enregistrement d'une modification se déroule en deux temps. On place d'abord dans l'index les fichiers concernés :

```
$ Terminal  
$ git add scenes/01_connexite/connexe_vs_arcs.py  
$ git add src/utils/colors.py
```

Puis on crée le *commit* proprement dit en lui associant un message descriptif :

```
$ Terminal  
$ git commit -m "Affine la scene ConnexeVsArcs et harmonise la palette"
```

Important

Un *commit* ne doit pas être compris comme une simple sauvegarde de travail : il représente une *unité intellectuelle* identifiable. Un bon *commit* groupe des modifications qui ont un sens ensemble et peut être décrit en une phrase concise ; un mauvais *commit* agglomère des évolutions hétéroclites qu'il sera ensuite difficile de relire, de réviser ou de réverter.

6.3.4 Publication distante : `git push`

La commande `git push` envoie les *commits* locaux vers le dépôt distant :

```
$ Terminal  
$ git push origin main
```

Si l'on travaille sur une branche dédiée — ce qui est vivement recommandé pour

tout travail non trivial —, cette branche doit être explicitement publiée :

```
$ Terminal  
$ git push origin scene-connexite
```

Le *push* ne crée pas de nouvelles modifications : il se borne à publier celles qui ont déjà été matérialisées localement par un *commit*.

6.3.5 Mise à jour locale : `git pull`

Symétriquement, la commande `git pull` récupère les modifications distantes et les fusionne dans la branche locale courante :

```
$ Terminal  
$ git pull origin main
```

Dans un projet collaboratif, cette opération doit être effectuée fréquemment pour éviter que le dépôt local ne diverge trop fortement du dépôt distant, car un *pull* tardif accroît mécaniquement le risque de conflits de fusion.

6.3.6 Branches de travail

Plutôt que de modifier directement la branche principale, il est recommandé de créer une branche dédiée à chaque tâche :

```
$ Terminal  
$ git checkout -b scene-connexite
```

Dans le cadre de *topo-manim*, les branches de travail ont typiquement correspondu à l'ajout d'une nouvelle scène (*scene-baire*), à la réécriture d'un module de la bibliothèque (*refactor-colors*) ou à une amélioration du rapport (*rapport-v2*). Cette discipline a permis à chacun des quatre contributeurs d'avancer en parallèle sans redouter l'interférence des autres, et de ne solliciter la branche principale qu'au moment des fusions contrôlées.

6.4 Application au dépôt *topo-manim*

Concrètement, le flux de travail que nous avons adopté se décompose en six étapes successives :

1. mise à jour du dépôt local par `git pull` ;
2. création d'une branche de travail dédiée à la notion étudiée ;

3. modification d'une scène du répertoire `scenes/` ou d'un module réutilisable de `src/`;
4. vérification de la cohérence du dépôt, au minimum par `make check` et `make test`, au mieux par le rendu effectif de la scène modifiée;
5. création d'un *commit* décrivant précisément l'évolution effectuée;
6. publication de la branche sur *GitHub* par `git push`, suivie d'une éventuelle revue par les autres contributeurs avant fusion.

Le *Makefile* du projet s'intègre naturellement dans ce processus : il formalise les commandes de vérification (`make check`, `make test`) et les commandes de rendu (`make <scene>`), si bien que le triptyque *modifier – vérifier – versionner* devient le régime normal de travail.

6.5 La gestion de versions comme discipline académique

Dans un contexte universitaire, la qualité de l'usage de *Git* ne se mesure pas à la seule maîtrise des commandes élémentaires : elle se mesure à la *discipline documentaire* que ces commandes rendent possible. Un dépôt bien tenu possède des *commits* lisibles dont le titre énonce clairement l'intention, des branches thématiques identifiables et un historique qui raconte, au lecteur qui en parcourt les journaux, le développement effectif du projet.

Pour un dépôt comme *topo-manim*, cette dimension est particulièrement importante. Le travail ne porte pas seulement sur du code d'exécution, mais aussi sur des objets pédagogiques et scientifiques dont la cohérence doit être maintenue dans le temps. Conserver proprement l'histoire des scènes, des conventions chromatiques, des modules réutilisables et du rapport écrit ne relève donc pas d'un souci technique accessoire : c'est un élément constitutif de la rigueur mathématique du projet.

Étude de cas : analyse approfondie de la scène `borel_lebesgue.py`

7.1 Un cas d'étude exemplaire

Parmi les cinq scènes du dépôt, le fichier `scenes/02_compacite/borel_lebesgue.py` se prête particulièrement à une étude de cas détaillée, pour au moins trois raisons. D'abord, il traite de la notion de **compacité**, qui occupe dans le cours de topologie métrique une place centrale : elle y joue le rôle de pivot entre les arguments séquentiels, hérités de l'analyse élémentaire, et les arguments de recouvrement, spécifiques de la pensée topologique. Ensuite, il articule en un seul scénario *plusieurs formulations* de cette notion — compacité séquentielle, Borel–Lebesgue, Heine–Borel, lemme de Lebesgue — si bien qu'il offre un condensé fidèle des grandes manières d'envisager la compacité. Enfin, par son volume raisonnable (environ 280 lignes), il se prête à une analyse ligne à ligne qui serait malaisée sur une scène plus massive.

7.2 Rappels mathématiques

Avant d'aborder l'analyse proprement dite, il convient de rappeler les énoncés centraux que la scène cherche à illustrer. Nous reprenons ici les définitions et théorèmes tels qu'ils figurent dans le *Mémo de topologie* [1].

Compacité séquentielle

Soit (X, d) un espace métrique. On dit que X est **séquentiellement compact** si toute suite d'éléments de X admet une sous-suite convergente dans X .

Théorème de Borel–Lebesgue

Un espace topologique X est **compact** si et seulement si tout recouvrement ouvert de X admet un sous-recouvrement fini. Dans un espace métrique, cette définition coïncide avec la compacité séquentielle.

Théorème de Heine–Borel

Une partie de $\mathbb{R}^n$ est compacte si et seulement si elle est à la fois **fermée** et **bornée**.

Lemme de Lebesgue

Soit (X, d) un espace métrique compact et $(U_i)_{i \in I}$ un recouvrement ouvert de X . Alors il existe un réel $\alpha > 0$, appelé **nombre de Lebesgue** du recouvrement, tel que toute boule $B(x, \alpha)$ soit contenue dans l'un au moins des U_i .

La scène ne se limite pas à rappeler ces énoncés : elle cherche à faire voir pourquoi ces formulations, en apparence différentes, relèvent d'une même idée directrice. Un ensemble compact est, intuitivement, un ensemble *sur lequel on ne peut pas perdre le contrôle en passant à l'infini* : ni par une suite qui s'échapperait, ni par une famille d'ouverts qu'il faudrait actualiser en des points de plus en plus proches. Toutes les formulations de la compacité sont autant de manières de dire cette chose.

7.3 Structure logicielle et imports

Le fichier définit une unique classe `BorelLebesgue`, héritée de `Scene`, et expose en préambule les imports nécessaires. On remarquera que ces imports reflètent exactement l'organisation architecturale du dépôt : les types graphiques viennent de `manim`, les constantes de temporisation de `src.config`, et la palette chromatique sémantique de `src.utils.colors`.

```

1  # -*- coding: utf-8 -*-
2  """Compacité et théorème de Borel-Lebesgue."""
3
4  from __future__ import annotations
5
6  import numpy as np
7  from manim import (
8      Circle, Create, Dot, FadeIn, FadeOut, Line, MathTex,
9      NumberLine, Scene, SurroundingRectangle, Text, VGroup, Write,
10     DOWN, ORIGIN, RIGHT, UP,
11 )
12
13 from src.config import DEFAULT_WAIT, SHORT_WAIT
14 from src.utils.colors import (
15     COVER_COLORS, DIM_COLOR, EPSILON_COLOR,
16     HIGHLIGHT_COLOR, OPEN_SET_COLOR, TEXT_COLOR,

```

```

17 )
18
19
20 class BorelLebesgue(Scene):
21     """Scène sur la compacité métrique et les recouvrements ouverts."""
22
23     def construct(self):
24         self.section_titre()
25         self.section_def_compacite()
26         self.section_recouvrement()
27         self.section_segment_compact()
28         self.section_non_compact()
29         self.section_lebesgue()

```

La méthode `construct` est d’une brièveté exemplaire : elle ne contient rigoureusement rien d’autre que l’appel séquentiel de six méthodes de section. Chacune de ces méthodes porte, pour toute sa durée, l’entière responsabilité d’une idée mathématique. Cette discipline présente une valeur méthodologique réelle : elle permet de faire correspondre chaque bloc de code à une unité pédagogique identifiable, rend le fichier lisible même pour un lecteur qui n’a pas participé à son écriture, et autorise la réorganisation du scénario par simple permutation d’appels de méthodes. Nous avons adopté ce patron pour toutes les scènes du dépôt : c’est devenu, à l’usage, un véritable *style maison*.

7.4 Analyse détaillée des sections

Chacune des six méthodes mérite un commentaire, car elle incarne un geste mathématique différent.

7.4.1 Section 1 : le titre

La méthode `section_titre` ouvre la scène par un titre — « Compacité et Borel-Lebesgue » — accompagné d’un sous-titre. Apparemment anecdotique, cette ouverture joue en réalité un rôle important : elle annonce le thème, installe l’attention du spectateur et cadre la narration qui va suivre.

7.4.2 Section 2 : la compacité séquentielle

La méthode `section_def_compacite` introduit la compacité séquentielle. Une droite graduée (`NumberLine`) y sert de support pour représenter les termes d’une suite de

points ; une valeur d'adhérence est distinguée visuellement par une couleur de mise en évidence ; certains points sont ensuite marqués comme appartenant à une sous-suite convergente. La scène ne démontre pas le théorème au sens formel : elle rend sensible, par la concentration visible des points autour d'une valeur, le mécanisme fondamental selon lequel, au sein d'une suite infinie dans un compact, une structure d'accumulation subsiste nécessairement.

7.4.3 Section 3 : le recouvrement

La méthode `section_recouvrement` effectue un changement de point de vue décisif. Le segment $[0, 1]$ y est recouvert par une famille d'ouverts figurés par douze disques colorés — la variété des couleurs provient de la liste `COVER_COLORS` importée depuis la palette du projet — puis seuls quelques-uns de ces ouverts sont conservés, les autres s'effaçant progressivement. La scène matérialise ainsi l'idée de *sous-recouvrement fini*. Le point essentiel n'est pas seulement que le segment est couvert : c'est qu'il *demeure* couvert après réduction à un nombre fini d'ouverts. L'animation fait sentir cette propriété mieux qu'aucune démonstration formelle ne pourrait le faire à un non-spécialiste.

7.4.4 Section 4 : le segment compact

La méthode `section_segment_compact` rattache l'observation précédente au théorème de Heine–Borel. Le segment $[0, 1]$ y est présenté comme fermé et borné dans $\mathbb{R}$, donc compact. Cette section joue un rôle de *synthèse intermédiaire* : elle reconnecte le phénomène visuel du recouvrement à un critère théorique standard du cours et permet au spectateur d'ancrer ce qu'il vient de voir dans un énoncé déjà connu.

7.4.5 Section 5 : l'exemple non compact

La méthode `section_non_compact` propose un contraste avec l'ensemble $]0, 1[$. Une suite de la forme $x_n = 1/n$ y est représentée, tendant vers 0 qui n'appartient pas à l'ensemble ; puis l'on exhibe explicitement une famille d'ouverts $(U_n)_{n \geq 1}$ — typiquement $U_n = (1/(n+2), 2)$ — qui couvre $]0, 1[$ sans admettre de sous-recouvrement fini, car en retirer ne serait-ce qu'un ouvert laisserait à découvert un voisinage de 0. Cette section, délibérément placée après la section sur le segment compact, introduit non seulement un exemple positif mais aussi un *mécanisme d'échec* de la compacité : elle montre comment la compacité peut « rater » et, par contraste, ce qu'elle exige effectivement.

7.4.6 Section 6 : le lemme de Lebesgue

La méthode `section_lebesgue` présente enfin le lemme de Lebesgue. La scène montre qu'à partir d'un recouvrement ouvert d'un compact, il est possible de choisir une taille uniforme $\alpha > 0$ telle que toute boule $B(x, \alpha)$ soit contenue dans l'un au moins des ouverts du recouvrement. Cette dernière section est particulièrement importante : elle fait apparaître une propriété plus fine que la compacité stricto sensu, à savoir l'existence d'un *contrôle uniforme local*. C'est précisément ce contrôle qui sous-tend, par exemple, la démonstration du théorème de Heine sur l'uniforme continuité des fonctions continues sur un compact.

7.5 L'inscription du fichier dans l'architecture du dépôt

Le fichier `borel_lebesgue.py` relève directement de la couche de **scénarisation** du dépôt. Placé dans `scenes/02_compacite/`, il définit une scène exécutable par *Manim* et le `Makefile` lui associe explicitement la cible `borel_lebesgue`, ce qui l'intègre sans ambiguïté dans la chaîne officielle de rendu du projet. Son exécution est donc aussi simple que la commande :

```
$ Terminal
$ make borel_lebesgue
```

Le fichier mobilise ensuite plusieurs composants de la couche de **mutualisation**. Il importe `DEFAULT_WAIT` et `SHORT_WAIT` depuis `src.config`, ce qui aligne son rythme narratif sur celui des autres scènes du dépôt ; il importe la palette de couleurs centralisée `COVER_COLORS`, `EPSILON_COLOR`, `HIGHLIGHT_COLOR`, `OPEN_SET_COLOR` et `DIM_COLOR` depuis `src.utils.colors`, ce qui garantit qu'un ouvert aura ici la même couleur que partout ailleurs dans le projet. Ce dernier point n'est pas accessoire : la constance chromatique est une promesse tacite faite au spectateur, qui peut ainsi investir progressivement un vocabulaire visuel stable.

On remarquera en revanche que la scène n'invoque pas la classe `Covering` définie dans `src/objects/coverings.py`, et qu'elle préfère construire à la main ses propres disques de recouvrement à l'aide de `Circle`. Cette observation, déjà soulevée au chapitre 2, est méthodologiquement révélatrice : elle montre que la structure modulaire du dépôt, bien qu'amorcée, n'est pas encore pleinement réinvestie dans toutes les scènes finales. Une évolution naturelle du projet consisterait précisément à réécrire `section_recouvrement` en s'appuyant sur `Covering.extract_subcover`, ce qui allégerait le code et renforcerait la cohérence architecturale.

7.6 Logique pédagogique : une montée en abstraction ordonnée

La qualité principale de cette scène ne tient pas à la virtuosité visuelle de ses animations — qui sont volontairement sobres — mais à la rigueur de sa logique pédagogique. Le fichier ne juxtapose pas six figures autonomes : il organise une véritable *montée en abstraction*, dans laquelle chaque section prépare intellectuellement la suivante.

- Le **premier niveau** est *séquentiel* : on part d'une suite et d'une sous-suite convergente, phénomène concret encore familier à l'étudiant d'analyse élémentaire.
- Le **deuxième niveau** est *topologique* : la compacité est reformulée en termes de recouvrements ouverts, ce qui déplace le regard d'une propriété « point par point » vers une propriété globale de l'espace.
- Le **troisième niveau** est *structurel* : avec Heine–Borel, la scène relie cette propriété globale à un critère géométrique simple dans $\mathbb{R}^n$ — fermé et borné — qui permet de reconnaître la compacité d'un ensemble sans avoir à manipuler explicitement des recouvrements.
- Le **quatrième niveau** est *critique* : l'exemple $]0, 1]$ montre que la compacité n'est pas une propriété automatique des sous-ensembles bornés et qu'elle repose essentiellement sur le caractère fermé.
- Le **cinquième niveau** est *uniforme* : le lemme de Lebesgue fait apparaître une version plus fine de la compacité, exprimée en termes de contrôle uniforme sur les boules, et dont découlent plusieurs énoncés classiques d'uniforme continuité.

Une telle progression est particulièrement réussie, car elle n'avance ni par accumulation désordonnée d'énoncés, ni par pure rhétorique visuelle. Chaque section a une fonction cognitive identifiable, prépare les suivantes et referme une question que les précédentes avaient ouverte. C'est en cela que cette scène nous semble exemplaire : elle fait voir, mieux que ne le ferait un cours écrit, la *hiérarchie conceptuelle* qui unit les différentes formulations de la compacité.

7.7 Pourquoi ce fichier est un bon exemple

Le choix de `borel_lebesgue.py` comme étude de cas se justifie dès lors par quatre raisons convergentes.

1. Il traite un objet mathématique *central* du cours de topologie métrique, à savoir la compacité, dont la maîtrise conditionne de nombreux résultats ultérieurs d'analyse.
2. Il articule plusieurs *formulations du même concept* au sein d'un seul fichier, ce qui en fait un condensé fidèle de la richesse de la notion.

3. Il mobilise *effectivement* la structure générale du dépôt — configuration partagée, palette chromatique, organisation en sections — tout en révélant, avec une franchise utile, les marges de factorisation encore possibles.
4. Il offre une *progression pédagogique claire* qui va du concret vers l'abstrait, et qui peut servir de modèle à la conception de nouvelles scènes.

À ce titre, `borel_lebesgue.py` n'est pas seulement une scène parmi d'autres : il peut être lu comme un *manifeste* de la philosophie générale du projet. Il montre par l'exemple comment un dépôt d'animations mathématiques peut servir, au-delà de la simple illustration d'un cours, à organiser visuellement une chaîne d'idées théoriques cohérentes dans laquelle chaque étape éclaire les précédentes et prépare les suivantes.

Conclusion et perspectives

8.1 Bilan du travail accompli

Ce projet nous aura conduits, en quelques mois, depuis la simple découverte de la bibliothèque *Manim* jusqu'à la constitution d'un dépôt logiciel opérationnel, couvrant trois grands chapitres du cours de topologie métrique de licence 3. Au terme du travail, *topo-manim* rassemble cinq scènes pédagogiques exécutables par une commande `make`, une bibliothèque interne d'environ mille lignes de code Python articulée en quatre sous-modules, un `Makefile` unifié et un ensemble minimal de tests d'intégrité. Le tout est placé sous contrôle de version *Git*, hébergé sur *GitHub* et documenté par un `README.md` ainsi que par le présent rapport.

Sur le plan *mathématique*, nous estimons avoir couvert fidèlement les notions fondamentales attendues : la connexité et son contre-exemple canonique $\sin(1/x)$, la compacité dans ses cinq formulations complémentaires — séquentielle, Borel–Lebesgue, Heine–Borel, contre-exemple, lemme de Lebesgue — et la complétude à travers le théorème de Baire et son application à la non-dénombrabilité de $\mathbb{R}$. Chacune de ces notions est illustrée par une scène *Manim* dont le scénario épouse la progression du cours de F. Le Roux et F. Klopp [1].

Sur le plan *logiciel*, le dépôt concrétise une intention architecturale claire : séparer rigoureusement les briques réutilisables des contenus pédagogiques, centraliser les paramètres globaux et la palette chromatique, organiser chaque scène selon un patron sectionné, et normaliser les commandes d'exécution au moyen d'un `Makefile`. Cette architecture est, dans son principe, solide ; elle n'est pas encore uniformément investie par les scènes, mais elle constitue l'infrastructure nécessaire à l'enrichissement futur du projet.

Sur le plan *méthodologique*, nous avons appris à concevoir une animation mathématique non comme un simple dessin animé, mais comme une *phrase visuelle* dont la syntaxe est la chronologie des opérations et dont la sémantique repose sur un vocabulaire graphique stable. Nous avons également mesuré l'importance d'un flux de travail *Git / GitHub* discipliné pour la coordination d'un projet à plusieurs contributeurs.

8.2 Difficultés rencontrées et enseignements

Le principal obstacle rencontré tient à la courbe d'apprentissage de *Manim* elle-même. Si le modèle de programmation — une classe, une méthode `construct`, une suite d'appels à `self.play` — est rapidement assimilable, la richesse des primitives graphiques et la subtilité des animations composées demandent un temps d'expérimentation non négligeable avant de pouvoir produire des scènes véritablement lisibles. Cette expérimentation s'est faite par essais et erreurs successifs, et nous a parfois conduits à réécrire entièrement des scènes dont la première version ne rendait pas l'idée mathématique avec suffisamment de clarté.

Une seconde difficulté, d'ordre conceptuel, a porté sur la *fidélité mathématique*. Il est tentant, pour rendre une idée visuellement frappante, de simplifier au-delà de ce que le formalisme autorise. Nous nous sommes efforcés, autant que possible, de ne pas compromettre la rigueur des énoncés affichés : chaque définition, chaque théorème visible à l'écran a été vérifié par rapport au *Mémo de topologie*. Ce souci nous a parfois conduits à renoncer à une animation esthétiquement séduisante au profit d'une version plus austère mais mathématiquement exacte.

8.3 Perspectives d'extension

Plusieurs directions d'extension du projet nous paraissent naturelles.

1. **Compléter le réinvestissement de la bibliothèque interne.** Les classes `MetricSpace`, `TopologicalSet`, `Path`, `Covering` ainsi que les classes d'animation `EpsilonDeltaAnimation`, `SequenceConvergence`, `ContinuousDeformation` sont prêtes mais pas encore systématiquement utilisées par les scènes finales. Un travail de refactorisation consisterait à les intégrer aux scènes existantes, ce qui réduirait la quantité de code spécifique et renforcerait la cohérence architecturale.
2. **Enrichir la couverture mathématique.** Plusieurs notions du cours n'ont pas encore été illustrées : les espaces normés et leurs topologies, la continuité uniforme, les espaces compacts produits, la convexité, ou encore les espaces de Banach. Chacune de ces notions se prête à une scène *Manim* indépendante qui viendrait s'ajouter aux cinq scènes existantes.
3. **Intégrer des narrations sonores.** *Manim* permet l'intégration de commentaires audio synchronisés avec les animations, ce qui rendrait le dépôt exploitable comme ressource pédagogique autonome, sans nécessiter d'accompagnement par un enseignant.
4. **Formaliser une suite de tests visuels.** Les tests actuels se bornent à vérifier l'intégrité des imports. Il serait possible d'automatiser le rendu d'une image témoin à certains instants clefs et de la comparer à une référence, ce qui détecterait

les régressions visuelles introduites par des modifications du code.

8.4 Mot de la fin

Au terme de ce travail, nous sommes convaincus que la visualisation mathématique, loin d'être un simple ornement pédagogique, constitue un véritable *levier cognitif* pour l'apprentissage de la topologie. Les notions d'ouvert, de compact, de complet ne se laissent pas saisir en un seul regard : elles exigent que l'on construise patiemment un imaginaire géométrique capable de les porter. La bibliothèque *Manim* offre, pour cette construction, un outil d'une puissance remarquable, dont nous espérons avoir proposé, avec *topo-manim*, une utilisation rigoureuse et durable.

Nous formons le vœu que ce dépôt puisse servir, dans les années à venir, de point de départ à d'autres étudiants désireux d'illustrer à leur tour une notion mathématique par une animation claire, cohérente et fidèle — et qu'il contribue ainsi, modestement, à faire voir ce que la topologie s'emploie à penser.

Bibliographie

- [1] F. Le Roux et F. Klopp. *Mémo de topologie*. Document de cours accompagnant le module 3M260 de la licence de mathématiques, Sorbonne Université, 2021.
- [2] G. Sanderson. *3Blue1Brown*. Chaîne YouTube de vulgarisation mathématique. <https://www.youtube.com/3Blue1Brown>.
- [3] The Manim Community Developers. *Manim Community Edition — Documentation*. Documentation officielle du projet *Manim*. <https://docs.manim.community/>.
- [4] J. R. Munkres. *Topology*. Prentice Hall, 2^e édition, 2000.
- [5] N. Bourbaki. *Éléments de mathématique. Topologie générale, chapitres 1 à 4*. Springer, nouvelle édition, 2007.